

SIMATS ENGINEERING

**SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105**

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – APPLICATION BASED QUESTIONS

Course Code:	<u>DSA0501</u>
Course Title:	<u>Query Processing using data analytics</u>
CO Assessed:	CO1
Assessment:	Tool 2 – Application Based Questions
Weightage:	_____
Total Marks:	20 Marks

CO1 Statement: _____

Student Name: S.Nandini_____ Reg. No.: 19243030_____

Section/Batch: _____ Date of Submission: _____

_____ Faculty In-charge: _____

_____ Submission Mode: Individual

Code of Conduct

I S.Nandini(192473030) certify that this submission is my original work and have adhered to all academic integrity guidelines.

Signature:

Criteria	Marks
Understanding of the Problem	4
Application of Concepts	4
Program Design / Algorithm	4
Correctness of Solution & Output	4
Explanation / Interpretation of Results	4
Total	20

O1 – AT2 – Application Based Questions

Sample Questions

Product Inventory Processing (10 Marks) An online retail company receives product information from suppliers in XML format. The XML file contains product names, categories, prices, and stock quantities. Write a Python program to parse the XML document, identify products with missing or invalid prices, correct the data wherever possible, and export the cleaned information into a CSV file for inventory management.

Hospital Patient Data Validation (10 Marks) A hospital stores patient information in a CSV file containing patient IDs, names, ages, blood groups, and contact numbers. Some records contain missing values, invalid ages, and duplicate patient IDs. Write a Python program to read the dataset, identify and correct data quality issues, remove duplicate records, and generate a validated dataset that can be imported into the hospital database.

Question 1: Product Inventory Processing using XML

Product Inventory Processing

Objective

The objective of this program is to read product information from an XML document, identify products that contain missing or invalid prices, automatically correct the data wherever possible, and export the cleaned information into a CSV file. This ensures accurate inventory records for the online retail company.

Problem Statement

An online retail company receives product details from multiple suppliers in XML format. Due to inconsistent data entry, some products have missing prices, invalid price values (negative or non-numeric), or incomplete information.

The task is to:

- Read the XML file.
- Parse product information.
- Detect missing or invalid prices.

- Correct invalid values wherever possible.
 - Save the cleaned data into a CSV file.
-

Algorithm

1. Import the required libraries (`xml.etree.ElementTree` and `csv`).
2. Load the XML file.
3. Parse all product records.
4. Extract:
 - Product Name
 - Category
 - Price
 - Stock Quantity
5. Validate the price.
6. If the price is:
 - Missing
 - Empty
 - Negative
 - Non-numericreplace it with **0.0**.
7. Store all cleaned records.
8. Write the cleaned data into a CSV file.
9. Display a success message.

Python Program

```
import xml.etree.ElementTree as ET
```

```
import csv
```

```
tree = ET.parse("products.xml")
```

```
root = tree.getroot()
```

```
cleaned_data = []
```

```
for product in root.findall("product"):
```

```
    name = product.find("name").text
```

```
    category = product.find("category").text
```

```
    stock = product.find("stock").text
```

```
price_element = product.find("price")

try:

    if price_element is None or price_element.text is None:

        raise ValueError

    price = float(price_element.text)

    if price < 0:

        raise ValueError

except:

    price = 0.0

cleaned_data.append([name, category, price, stock])

# Export cleaned data to CSV

with open("cleaned_inventory.csv", "w", newline="") as file:

    writer = csv.writer(file)

    writer.writerow(["Product Name",

                     "Category",

                     "Price",

                     "Stock"])

    writer.writerows(cleaned_data)
```

```
print("Inventory cleaned successfully.")

print("Data exported to cleaned_inventory.csv")
```

Output

```
... Inventory cleaned successfully.
    Data exported to cleaned_inventory.csv
```

Cleaned dataset:

...	Product Name	Category	Price	Stock
0	Laptop	Electronics	65000.0	25
1	Headphones	Electronics	0.0	40
2	Chair	Furniture	0.0	15
3	Table	Furniture	3500.0	8

Explanation

The program uses Python's **xml.etree.ElementTree** library to parse the XML document. Each product is examined individually, and the price field is validated. Missing, empty, negative, or non-numeric prices are replaced with **0.0** to maintain data consistency. Finally, the cleaned records are exported into a CSV file, making them suitable for inventory management systems.

Advantages

- Detects invalid prices automatically.
 - Prevents incorrect inventory calculations.
 - Converts XML data into CSV format.
 - Easy to maintain and extend.
 - Suitable for real-world retail inventory systems.
-

Applications

- E-commerce inventory management
 - Warehouse management systems
 - Retail analytics
 - Product catalog management
-

Conclusion

The program successfully validates product information received in XML format, corrects invalid pricing data, and exports the cleaned records into a CSV file. This improves data accuracy and ensures reliable inventory management.

Question 2: Hospital Patient Data Validation using CSV

Hospital Patient Data Validation

Objective

The objective of this program is to validate hospital patient records stored in a CSV file by identifying missing values, correcting invalid data, removing duplicate patient IDs, and generating a cleaned dataset suitable for importing into the hospital database.

Problem Statement

Hospitals store patient information in CSV files. Due to manual data entry, the dataset may contain:

- Missing values
- Invalid ages
- Duplicate patient IDs

- Incorrect contact numbers

The task is to clean and validate the dataset before importing it into the hospital management system.

Algorithm

1. Import the CSV module.
2. Read the patient dataset.
3. Maintain a set of processed Patient IDs.
4. For each record:
 - Skip duplicate Patient IDs.
 - Replace missing names with **Unknown**.
 - Validate age.
 - Replace invalid ages with **0**.
 - Validate blood group.
 - Replace invalid blood groups with **Unknown**.
 - Validate contact number.
 - Replace invalid contact numbers with **Not Available**.
5. Store cleaned records.
6. Export the validated dataset.
7. Display success message.

Python Program

```
import csv

valid_blood_groups = {
    "A+", "A-",
    "B+", "B-",
    "AB+", "AB-",
    "O+", "O-"
}

cleaned_records = []

patient_ids = set()

with open("patients.csv", "r") as file:

    reader = csv.DictReader(file)

    for row in reader:

        pid = row["PatientID"]

        if pid in patient_ids:
            continue
```

```

patient_ids.add(pid)

name = row["Name"].strip()

if name == "":
    name = "Unknown"

try:
    age = int(row["Age"])

    if age < 0 or age > 120:
        age = 0

except:
    age = 0

blood = row["BloodGroup"]

if blood not in valid_blood_groups:
    blood = "Unknown"

contact = row["Contact"]

if len(contact) != 10 or not contact.isdigit():
    contact = "Not Available"

cleaned_records.append({
    "PatientID": pid,
    "Name": name,
    "Age": age,
    "BloodGroup": blood,
    "Contact": contact
})

with open("validated_patients.csv",
        "w",
        newline="") as file:

    fields = ["PatientID",
              "Name",
              "Age",
              "BloodGroup",
              "Contact"]

    writer = csv.DictWriter(file,
                           fieldnames=fields)

    writer.writeheader()

    writer.writerows(cleaned_records)

print("Patient records validated successfully.")
print("Validated dataset exported.")

```


Output:

```
... Patient records validated successfully.
Validated dataset exported.
```

Cleaned Dataset:

"

	PatientID	Name	Age	BloodGroup	Contact
0	101	Ravi	25	B+	9876543210
1	102	Priya	0	O+	Not Available
2	103	Amit	0	A+	9876543218
3	104	Unknown	30	AB+	9876543222
4	105	John	0	Unknown	9876543333

Next steps: [Generate code with df](#) [New interactive sheet](#)

Explanation

The program reads patient records from a CSV file and validates each field. Duplicate patient IDs are removed using a Python set. Missing names are replaced with **Unknown**, invalid ages are corrected to **0**, unsupported blood groups are replaced with **Unknown**, and invalid contact numbers are marked as **Not Available**. Finally, the cleaned data is exported into a validated CSV file that can be safely imported into the hospital database.

Advantages

- Eliminates duplicate patient records.
- Detects missing values.
- Validates age and blood group.
- Checks contact number format.
- Produces a database-ready dataset.
- Improves hospital data quality.

Applications

- Hospital Information Systems (HIS)
 - Electronic Health Records (EHR)
 - Medical Data Management
 - Healthcare Analytics
 - Patient Registration Systems
-

Conclusion

The program successfully validates hospital patient records by correcting missing and invalid data, removing duplicate entries, and generating a clean CSV dataset. This improves the accuracy, consistency, and reliability of patient information, ensuring that the hospital database contains high-quality records for efficient healthcare management.